技术白皮书 · V1.3 · 性能工程实证

把一个 504 从系统里移出去

私有化知识库管理控制台上，管理员点一次「刷新译文状态」，稳定收到 `HTTP 504 Gateway Time-out`。 本文用受控实验把原因从「系统太慢」收窄到一个可量化的具体缺陷——**每次查询对同一文件执行 4 次读取**—— 再说明为什么把它优化成 1 次也不够，以及为什么真正的修复是让请求**不再等待扫描。** 所有数字均可由随文脚本复算。

2026-10-29 | Lee Teong Hoe · Hoelee Enterprise | 受控实验 · 三次独立全跑 | 请求路径 0.3 µs | 附复算脚本与原始读数

01 摘要

这个缺陷的危险之处不是它慢，而是**它在开发机上根本不构成故障。** 本地 NVMe 上，四重读取的朴素实现只比单次遍历慢 1.5–1.9 倍；一旦存储换成网络挂载卷， 同一份代码按读取次数线性放大到十几秒，才越过外层网关的响应头上限。这是一条「本地全绿、生产 504」的标准路径。

朴素实现 · 冷	单次遍历 · 冷	SWR 缓存命中
433.5 ms	230.0 ms	0.3 µs
960 次整文件读取 / 360 文件语料	只快 1.5–1.9×，仍是 8 秒级	请求路径零 I/O

增量刷新 · 1 篇变更
14.7 ms
只读 1 个文件，复用 239 条

1. **四重读取放大（Read Amplification）**：单次状态查询对同一文件执行 4 次 `read_text()` 与约 3 次 YAML frontmatter 反序列化。240 篇源笔记 → **960 次**整文件读取。
2. **该缺陷与存储时延耦合，不与 CPU 耦合**：本地磁盘上 433.5 ms vs 230.0 ms（三次全跑分别 1.61 / 1.50 / 1.89，按中位相除 1.88×）；按每次读取代价 20 ms 投影，朴素实现 **18.5 s**、单次遍历 **7.9 s**——后者不再 504，但对管理员仍是一块卡死 8 秒的控制台。

3. **因此正确的修复不是「把扫描写快一点」**：而是把扫描**移出请求路径**。请求退化为一次内存字典读取，陈旧窗口由 60 秒 TTL 与后台单飞刷新约束。

> 重构后的验收不是「快了多少倍」，而是**请求路径上是否还有文件系统 I/O**。 顺带必须守住一条：缓存化最容易悄悄削掉的不是延迟，而是业务能力——本方案的 `ready / outdated / pending` 三态分类被列为与延迟同等的验收项。

---

02 故障现场与读取放大

2.1 请求经过的每一层

故障请求从浏览器出发，穿过四层服务端组件：入口网关终止 TLS 并转发，鉴权网关注入身份头，后端反代分流静态与 API，最后由应用同步遍历两个目录——源笔记与译文。三层的超时预算写在每个方块右侧。

终端浏览器 / 管理控制台 `HTTPS :443`

▼ TLS 终止与路由转发

外层入口网关 · Traefik v3 `responseHeaderTimeout = 30s`

▼ internal docker network，转发前注入身份头

身份认证网关 · Forward-Auth `forward auth provider`

▼ reverse proxy header injection

后端反代 · Nginx（静态资源与 API 分流） `proxy_read_timeout = 60s`

▼ HTTP / unix socket

应用服务 · Flask 后端 `gunicorn --timeout 120s`

▼ 同步磁盘 I/O：源笔记 `videos/*.md` 与译文 `videos/*.en.md` 逐篇遍历

2.2 前端如何呈现这次失败

控制台用常规异步 Fetch 拉状态；504 被 `catch` 捕获后，横幅转为持久红色报错。前端本身没有问题——它只是忠实地把一个后端缺陷显示出来。

```
async function refreshTranslationStatus() {
  const banner = document.getElementById('trStatusBanner');
  banner.style.display = 'block';
  banner.textContent = '正在获取全库双语译文状态...';
  try {
    const res = await fetch('/api/translations/status');
    if (!res.ok) throw new Error('HTTP ' + res.status);
    const data = await res.json();
    renderTranslationStatus(data);
    banner.style.display = 'none';
  } catch (e) {
    // 此处高频捕获 Error: HTTP 504，横幅转为红色持久报错
    banner.textContent = '读取译文状态失败：' + e;
  }
}
```

2.3 缺陷定位：一次查询，四次读取

路由端点 `api_translation_status()` 调用 `list_status()`。原实现对每个文件重复读取：先读一次判断 locale，再读一次算哈希，又读一次取标题与分类，最后存在译文时再读译文。

```
def list_status(content_dir="src/content/videos", locale="zh"):
    results = []
    for p in Path(content_dir).glob("*.md"):
        if p.name.endswith(".en.md"):
            continue

        # 第 1 次 I/O + 解析：读取文件判断 locale
        raw = p.read_text(encoding="utf-8")
        meta, body = split_note(raw)
        if meta.get("locale") != locale:
            continue

        results.append(status(p))
    return results

def status(source_path):
    # 第 2 次 I/O：hash_of_note 内部再次 read_text + split_note
    src_hash = hash_of_note(source_path)

    # 第 3 次 I/O：又读一次以提取标题与分类
    meta, _ = split_note(source_path.read_text(encoding="utf-8"))

    # 第 4 次 I/O：存在 .en.md 时再读译文并比对
    target_path = source_path.with_name(f"{source_path.stem}.en.md")
```

```
    if target_path.exists():
        tgt_hash = hash_of_note(target_path)
```

放大效应：240 篇源笔记时，单次请求执行 240 × 4 = **960 次**整文件读取、约 720–960 次 YAML 反序列化。第 5 章的实测表明，这在本地磁盘上「只」是 1.5–1.9 倍差距，但在网络挂载卷上按读取次数线性放大。

---

03 超时预算倒置

多层代理的典型反模式：**下游的超时预算长于上游。**上游一旦先斩断连接，下游的响应就再没有人接收—— 算力已经花掉，结果无人收取。这种「孤儿响应」不会出现在任何一层自己的监控里：上游只记一条 504，下游只记一条 200。

代理层级	配置指令	设定值	行为特征
外层入口网关	`forwardingTimeouts.responseHeaderTimeout`	30 秒	超过 30s 未收到 upstream 响应头即掐断客户端连接并返回 504
中层反代	`proxy_read_timeout`	60 秒	继续等待后端应用，上限 60s
应用容器	`--timeout`	120 秒	仍在无感知地占用 worker、CPU 与存储带宽

物理证据：故障期间外层网关按配置在响应头上限处返回 504 并关闭客户端 TCP；应用容器的同一请求在网关放弃*之后才*返回 200。该请求的完整耗时无法从客户端观测（客户端已收到 504），因此本文**不给出无法由两侧日志对齐的精确秒数**——可作为证据的，是两侧日志里同一 `X-Request-ID` 的「网关已放弃 / 应用已完成」并存。

3.1 为什么单次遍历不足以修复

把 4 次读取压成 1 次，实测只带来 1.5–1.9 倍改善。按每次读取 20 ms 投影，冷启动扫描仍需 **7.9 s**：不再触发 504，但对使用者仍是一个卡死近 8 秒的控制台，且每次 TTL 到期都要再卡一次。

> **这条判据可以推广到所有「遍历文件系统」的接口**：只要扫描在请求路径上，优化能改善的是常数，改善不了的是「请求必须等扫描跑完」这一结构事实。

04 架构重构：SWR 内存引擎 + 单飞锁

目标结构只有一句话：**请求永远读内存，扫盘永远发生在后台。** 新鲜则直读；过期则抢占刷新权、后台线程扫描、本次请求仍返回上一版本；并发请求不排队等扫描，而是直接拿旧值。



4.1 核心实现

三件事在同一把锁下完成：缓存条目、签名表、刷新熔断标志。签名表 `mtime_map` 是增量能力的全部来源——签名未变的文件**零次** `read_text()`，直接复用上一轮结果。

🔍 展开 SWR 缓存、单飞锁与 mtime 差量引擎核心实现（Python，73 行）

```
import threading
import time
from pathlib import Path

_TTL = 60.0
_CACHE = {"data": None, "stamp": 0.0, "refreshing": False, "mtime_map": {}}
_LOCK = threading.Lock()        # 保护 _CACHE 本身
```

```
_COLD_LOCK = threading.Lock()   # 冷启动单飞：只允许一个请求扫盘

def get_status(content_dir, locale="zh"):
    with _LOCK:
        has_data = _CACHE["data"] is not None
        is_fresh = (time.time() - _CACHE["stamp"]) < _TTL
        if has_data and is_fresh:
            return _CACHE["data"]                       # A 新鲜：零 I/O
        if has_data and _CACHE["refreshing"]:
            return _CACHE["data"]                       # B 已有刷新在跑：返回陈旧
        if has_data:                                    # C 抢占刷新权后立即返回
            _CACHE["refreshing"] = True
            threading.Thread(target=_refresh_bg, args=(content_dir, locale),
                             daemon=True).start()
            return _CACHE["data"]
    with _COLD_LOCK:                                    # D 冷启动：排队而非惊群
        with _LOCK:
            if _CACHE["data"] is not None:
                return _CACHE["data"]
        return _refresh_now(content_dir, locale)

def _refresh_now(content_dir, locale):
    rows, new_map = _scan(content_dir, locale)
    with _LOCK:
        _CACHE["data"] = rows
        _CACHE["mtime_map"] = new_map   # data 与签名表必须同锁同写，否则读者会看到半新半旧
        _CACHE["stamp"] = time.time()
        _CACHE["refreshing"] = False
    return rows

def _refresh_bg(content_dir, locale):
    try:
        _refresh_now(content_dir, locale)
    except Exception:
        with _LOCK:
            _CACHE["refreshing"] = False   # 刷新失败不能让熔断标志永久卡住

def _scan(content_dir, locale):
    rows, new_map = [], {}
    for p in sorted(Path(content_dir).glob("*.md")):
        if p.name.endswith(".en.md"):
            continue
        st = p.stat()
        tgt = p.with_name(f"{p.stem}.en.md")
        ts = tgt.stat() if tgt.exists() else None
        sig = (st.st_mtime_ns, st.st_size,
               ts.st_mtime_ns if ts else None, ts.st_size if ts else None)
        seen = _CACHE["mtime_map"].get(p.name)
        if seen and seen[0] == sig:                     # 签名未变：0 次 read_text
```

```
            rows.append(seen[1])
            new_map[p.name] = seen
            continue
        raw = p.read_text(encoding="utf-8")
        meta, body = split_note(raw)
        if meta.get("locale") != locale:
            continue
        src_hash = hash_memory(meta, body)
        row = {"slug": p.stem, "title": meta.get("title", p.stem),
               "source_hash": src_hash, "status": "pending"}
        if ts:                                          # 三态：译文是否已过期
            tgt_meta, _ = split_note(tgt.read_text(encoding="utf-8"))
            row["status"] = ("ready" if (tgt_meta or {}).get("translatedFromHash") ==
src_hash
                             else "outdated")
        rows.append(row)
        new_map[p.name] = (sig, row)
    return rows, new_map
```

4.2 相对首版修正的三处真实缺陷

1. `mtime_map` **从「声明了但没用」变成真正的增量机制。**首版的后台刷新仍全量 `read_text()`，签名表从未被读取；现在以 `(st_mtime_ns, st_size)` 双端签名作为缓存键。实测：240 篇中仅 1 篇变更时只读 1 个文件。
2. **三态检测被恢复。**首版把「译文是否过期」退化成「译文文件存在吗」，直接删掉了核心业务能力。现在比较译文 frontmatter 里记录的 `translatedFromHash` 与当前源文件哈希，正确区分 `ready` / `outdated` / `pending`。

   > 这里纠正了一个我先写错的实现：拿**源文正文哈希**与**译文正文哈希**直接比对永远不可能相等——它们本来就是不同语言的文本。过期判定的依据必须是「译文生成时刻记录的源哈希」。这个错误由基准实验本身暴露：第一版语料跑出 0 篇 `ready`，才让人注意到底层判定立不住。

3. **冷启动惊群被封住。**首版在无缓存时直接全盘扫描，5 个并发请求会同时扫盘；现在冷启动走独立的 `_COLD_LOCK`，第一个请求扫盘，其余排队后直接命中结果。v1.2 另把 `data` 与 `mtime_map` 的写入并入同一临界区，避免读者拿到「新数据配旧签名表」的半新半旧状态。

4.3 多进程扩展：四条路径，按运维成本排序

上述状态是**进程内**的。当应用以 2–4 个 worker 运行时，每个 worker 各扫一次盘、各自维护 TTL。四条演进路径：

方案	跨进程延迟	外部依赖	适用判断
每 worker 独立缓存 + 抖动 TTL	`0`（本地内存）	无	worker ≤ 4、语料 ≤ 500 篇时最优：最坏重复 4 次扫描，请求路径依旧微秒级
tmpfs 原子快照 + `fcntl.flock`	`< 0.5 ms`	无	单机多 worker 且要求状态一致的零依赖方案；容器需显式挂载
SQLite（WAL 模式）状态表	`~1 ms`	无	需要跨重启保留状态或需要审计时
独立 Redis 实例	`~0.3 ms`	有	仅当系统已有 Redis 时划算；为一张状态表新增中间件不成立

容器化部署若采用 tmpfs 方案需注意：Docker 默认把它分配为 **64 MB**，而 1,000 篇文档的状态 JSON 快照约 200 KB，远低于该上限，因此默认值不构成否决理由。

本项目选择第一档：**不动架构、承认重复扫描。**理由是这项操作 QPS 极低（管理员手动触发），一致性要求是最终一致，60 秒 TTL 已足够吸收。加一个 Redis 会把「本地文件系统问题」变成「新的可用性依赖」——用更大的组件解决更小的问题。

`05` 受控复现实验

5.1 方法

- **语料**：程序化生成 240 篇源笔记 + 120 篇译文，共 360 个 `.md`、1,315,740 字节；每篇含 12 段正文与 YAML frontmatter。其中 60 篇译文记录了当前源哈希（`ready`）、60 篇源文件在翻译完成之后又被修改（`outdated`）、120 篇无译文（`pending`）。
- **环境**：本机 NVMe SSD，Windows 11，Python 3.14.9，单进程。**这不是生产环境**：本实验量化的是「读取放大」的相对代价，而非复刻 NAS 的绝对时延。
- **方法**：每种实现预热 1 轮后连续测量 5 轮取中位数；缓存命中延迟以 200 次计时取中位数与 P99；**整个实验独立全跑 3 次**，下表报告三次的中位数并给出极值区间。
- **复算与存档**：随本记录归档的是执行脚本、**2026-10-30 复制会话（B）的三份原始 JSON**，以及 5.4 节讨论过的那两份**修正前 v1.1 输出**。会话 A（即 5.2 表中数据）的三次全跑取自控制台转录、未逐次落盘——这正是 5.5 节要重跑一遍而不是直接断言的原因。脚本 stdout 为严格合法 JSON、不含本机路径，任何一次运行都可直接进 CI 比对。

5.2 实测读数（本机 NVMe，三次全跑）

指标	中位	三次全跑区间
朴素实现（4 次读 + 3 次 YAML 解析）· 冷	**433.5 ms**	333.8 – 452.1 ms
单次遍历实现（1 次读 + 1 次解析）· 冷	**230.0 ms**	222.6 – 280.9 ms
SWR 冷启动（含 `stat` 与签名建表）	**263.1 ms**	202.5 – 275.1 ms
SWR 缓存命中（请求路径）	**0.3 µs**	P99 2.8 – 7.8 µs
SWR 增量刷新（TTL 到期，1 篇变更）	**14.7 ms**	14.53 – 15.74 ms
三态分类正确性	**ready 60 / outdated 60 / pending 120**	三次完全一致

关键读数：朴素实现对单次遍历只有 **1.5–1.9×** 差距（三次全跑分别计算为 1.61 / 1.50 / 1.89；按上表中位相除为 **1.88×**，两者都远不到「4 次读 = 4 倍耗时」的直觉）。四倍读取放大在本地磁盘上不会被感知——这就是该缺陷能在开发机上长期存活的原因，也是「本地测试通过」不能作为性能证据的直接反例。

同时必须承认：**这台机器的抖动本身就有 ±20% 量级**（同一实现跨三次全跑）。因此本文只引用区间与中位数，不给出精确到 0.1 ms 的单一数值——那种精度在这类环境中并不存在。而 5.5 节实测的**会话间**漂移更大：19–27%。

5.3 增量路径的价值

增量刷新（14.7 ms）相对冷启动单次遍历（230.0 ms）是 **15.6×**（各次全跑 14.6–19.3×），相对朴素实现（433.5 ms）约 **29.5×**。这两个倍数都属于会话 A；2026-10-30 复制会话实测为 11.5× 与 20.3×（见 5.5），因此不作规格指标引用。但真正决定体验的不是这个倍数，而是**增量刷新是否在请求路径上**——本实现中它不在：它发生在后台线程，请求方拿到的是上一版本缓存。

5.4 网络卷时延投影（模型推演，非实测）

投影的做法不是「基线耗时 + 读取次数 × L」的算式外推，而是**把 `Path.read_text` 拦截成 `sleep(L)` 之后重跑同一份实现**，因此表内数字包含 Python 调用开销与 `sleep` 的实际粒度，不会与「基线 + 次数 × L」严丝合缝——读者若按那个算式复核，会看到几百毫秒的系统性偏差，那是模型的实现方式所致，不是排版错误。三次行分别对应 960 / 360 / 360 次 `read_text`（另加约 480 次 `stat`）与 2 次读取。**这是投影，不是 NAS 实测值。**

单次读取代价 L	朴素（960 次读）	单次遍历（360 次读）	SWR 冷启动	SWR 增量（1 篇变更 / 2 次读）
0 ms（实测）	433.5 ms	230.0 ms	263.1 ms	14.7 ms
5 ms	5,552 ms	2,347 ms	2,433 ms	18.5 ms
10 ms	9,654 ms	4,244 ms	4,239 ms	30.2 ms
20 ms	**18,508 ms**	7,898 ms	7,857 ms	36.9 ms

> **v1.2 修正的一处实验缺陷**：v1.1 表格最后一列在 0 ms 行填的是「1 篇变更」的实测值，而 5/10/20 ms 行填的却是「无变更、0 次读」的投影值，导致同一列出现「读取惩罚越大反而越快」的反物理现象。现已在投影模

> 块中**先 touch 1 篇再测**，四行测试条件完全同构：增量列随 L 单调上升（14.7 → 18.5 → 30.2 → 36.9 ms），增量恒为 2 次 `read_text`。

读法：朴素实现的耗时几乎正比于读取次数。当 L 达到 30 ms/read 量级时，仅 I/O 就达 960 × 30 ms = 28.8 s，加上解析开销即越过 30 秒响应头上限——与生产现象吻合。而 SWR 增量路径只读 1 个文件，在任何 L 下都停留在几十毫秒量级。

> **本投影的局限（必须承认）**：模型只对 `read_text()` 计费，未对 `stat()` 计费。在 SMB/NFS 上每次 `stat` 同样是一次往返，而增量路径的主要成本恰恰是约 480 次 `stat`。因此「SWR 增量 18–37 ms」在真实网络卷上会**明显偏乐观**：若每次 `stat` 往返 5 ms，增量刷新的真实代价约为 480 × 5 ms + 37 ms ≈ 2.4 s——仍在请求路径之外，不影响 504 结论，但不该被描述成「几十毫秒」。

5.5 跨会话复制实验（2026-10-30）

5.2 与 5.4 的每一个数字都来自会话 A。单次会话证明不了可复现性，因此第二天用同一套脚本、同一台机器、同一语料生成器又跑了三次：无 GPU 负载，无其它压测在跑。这三次的原始 JSON 随本记录归档。

复制实验	朴素 · 冷启动	单次遍历 · 冷启动	SWR 冷启动	缓存命中（中位 / P99）	增量刷新	朴素 ÷ 单次
第 1 次	461.7 ms	242.7 ms	293.7 ms	0.3 / 3.1 µs	29.51 ms	1.90×
第 2 次	517.1 ms	292.2 ms	333.1 ms	0.6 / 6.4 µs	25.42 ms	1.77×
第 3 次	559.5 ms	306.6 ms	357.2 ms	0.5 / 3.5 µs	25.50 ms	1.82×
中位数	**517.1 ms**	**292.2 ms**	**333.1 ms**	**0.5 / 3.5 µs**	**25.50 ms**	**1.77×**

四条结论，其中一条不太舒服：

- **结构完全复现。**缓存命中稳定在亚微秒量级（中位 0.5 µs、P99 3.5 µs），增量刷新每次都只读 1 个文件。**请求路径上零文件系统读取**——这正是整套重构存在的理由，也是全文唯一不依赖计时的结论。
- **冷启动的排序关系复现。**三次复制实验中，朴素实现的代价是单次遍历的 1.77–1.90 倍（会话 A 为 1.50–1.89 倍）。砍掉四分之三的读取，在本机换来的仍是不到 2 倍。
- **绝对值没有复现。**本次会话在朴素 / 单次 / SWR 冷启动三条路径上分别比会话 A 慢 19% / 27% / 27%。**会话间漂移比会话内的 ±20% 更大**——中位数是一次会话的属性，不是常量。这条正是本文用来质疑别人的论点，现在用回本文自己的表。
- **增量指标最不稳定，5.3 的倍数因此是全文最软的数字。**会话 A 为 14.7 ms，复制会话中位 25.5 ms（25.4–29.5）。它的代价主要由约 480 次元数据调用构成，而单次代价取决于缓存状态——正是 5.4 节承认「投影没为 stat 计费」的那一项。倍数在会话 A 是 15.6×、在这里是 11.5×；两次都超过 10×，两次的结论（扫描已移出请求路径）都成立。

复制实验本身还有**会话内**漂移：连续三次 461.7 → 517.1 → 559.5 ms，单调变慢。笔记本热降频下，基准测试是一个移动靶，换运行顺序也解决不了。本文把它报出来，而不是抹平。

06 工程结论与边界兜底

1. **性能缺陷与存储时延耦合时，本地基准会产生假阴性。**1.5–1.9× 的本地差距不足以让人报警；18.5 s 的网络卷代价才会。任何遍历文件系统的实现都必须以最慢存储上的读时延做容量规划，而不是以开发机为准。
2. **超时预算必须下游短于上游。**30s / 60s / 120s 的倒置不是「给了更多余地」，而是制造孤儿响应：上游已放弃，下游仍在烧算力。
3. **把慢操作移出请求路径，比把它优化快更有效。**请求路径从 433.5 ms 降到 0.3 µs，靠的不是算法，而是「这一秒不扫盘」。
4. **冷启动仍是最贵的一次扫描，必须预热。**本项目在容器启动时以一次性后台任务触发首次扫描，把唯一一次 263 ms（网络卷上可能数秒）的代价放在进程就绪之前，而不是落在第一个管理员点「刷新」的时刻。
5. **缓存化的最大风险是业务能力被静默削掉。**首版的退化（`outdated` 态消失）不会被任何性能测试发现，只会被用户晚几天发现。因此三态分类正确性被列为与延迟同等的验收项。
6. **缓存一致性只做服务端写穿，不暴露客户端强制刷新开关。**保存译文时由服务端 `invalidate(slug)` 局部更新；**不提供** `?force_refresh=1`——多一个客户端可控的强制刷新入口，就多一个绕过缓存打穿磁盘的攻击面。

> **尚未解决的边界（已知，未修）**：写穿失效依赖「保存译文」这一写入路径必然回调 `invalidate(slug)`。若译者通过 rsync 或编辑器直接落盘，缓存最长 60 秒读到旧状态。本项目接受这个最终一致窗口，不在 `GET` 路径上加 `stat` 探测——那会把刚移出请求路径的文件系统 I/O 又请回来。

07 参考与复算

07.1 所引工业标准与规范

八条来源，每条都对应正文里实际用到的一个机制——不是文献综述。缓存语义取自 HTTP 规范，刷新协调模式取自 Go 生态中已成标准实现的出处，每次操作的往返代价取自文件系统协议本身。

[1] Nottingham, M. *RFC 5861: HTTP Cache-Control Extensions for Stale Content*. IETF, 2010-05. `https://www.rfc-editor.org/rfc/rfc5861.html`

定义 `stale-while-revalidate`，即「先回旧值、后台刷新」的语义来源，本引擎实现的正是这一条。

[2] Fielding, R., Nottingham, M., Reschke, J. *RFC 9111: HTTP Caching*. IETF, 2022-06.

https://www.rfc-editor.org/rfc/rfc9111.html

新鲜期与年龄计算口径，60 秒 TTL 的术语与判定依据。

[3] Go Project. *Package singleflight（golang.org/x/sync/singleflight）*.
https://pkg.go.dev/golang.org/x/sync/singleflight

并发去重原语的标准表述：同一时刻的多个调用共享一次在途结果。

[4] Fitzpatrick, B., et al. *groupcache*.
https://github.com/golang/groupcache

单飞式缓存填充模式的出处，以及它要解决的缓存击穿（cache stampede）问题。

[5] Callaghan, B., Pawlowski, B., Staubach, P. *RFC 1813: NFS Version 3 Protocol Specification*. IETF, 1995-06.
https://www.rfc-editor.org/rfc/rfc1813.html

每一次文件操作都是一次独立的 RPC——这正是不按字节数、而按「操作次数」计时的依据。

[6] Noveck, D., ed. *RFC 8881: Network File System (NFS) Version 4.1 Protocol*. IETF, 2020-08.
https://www.rfc-editor.org/rfc/rfc8881.html

确认元数据操作与读操作具备同样的往返结构，因此 5.4 节的投影对 `stat` 是低估的。

[7] Microsoft. *MS-SMB2: Server Message Block (SMB) Protocol Versions 2 and 3*. Open Specifications.
https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-smb2/

本文所指的「网络挂载卷」形态：请求/响应帧走网络，按设计就是一次操作一次往返。

[8] Python Software Foundation. *pathlib — Path.stat() 与 os.stat()*. Python 3 文档.
https://docs.python.org/3/library/pathlib.html#pathlib.Path.stat

确认 `stat` 是独立于 `read_text()` 的元数据调用——这就是模型与真实 SMB 挂载之间的差额。

07.2 复算

本文所有数字取自三次独立全跑的原始 JSON 中位数字段，可由随文脚本在自有硬件上复算（允许 ±30% 硬件差异）。脚本 stdout 为严格合法 JSON、不含任何本机路径，可直接用于 CI 比对。

```
python swr_bench.py            # 语料生成 + 三种实现 + 投影模块，全跑约 40 秒
                               # 输出：严格 JSON（无本机路径），可直接 diff
```

07.3 引用格式

```
Lee Teong Hoe. 《把一个 504 从系统里移出去：分布式知识库多级反代超时排查
与异步 SWR 状态汇总架构重构》. 技术白皮书 v1.3. Hoelee Enterprise, 2026-10-30.
https://doc.hoelee.com/whitepaper-504-read-amplification-swr.html
```

复算件 · 执行脚本、三次复制实验原始 JSON、修正前 v1.1 输出：github.com/hoelee/swr-read-amplification-bench（MIT；正文 CC BY 4.0）

Hoelee Enterprise · 马来西亚中小企业数字化服务 · blog.hoelee.com

本文所述架构模式由作者工程实践沉淀；客户标识与生产阈值已做脱敏处理。文中投影表为模型推演而非生产实测，结论以其自身标注的边界为限。

技术白皮书 v1.3 · 2026-10-30 · 初版 v1.0 2026-10-28 · 数据口径经三轮独立审阅收敛

本页为私有文档中心，仅凭直链访问，不对外索引。